A Hybrid-Domain Floating-Point Compute-in-Memory Architecture for Efficient Acceleration of High-Precision Deep Neural Networks

Zhiqiang Yi, Yiwen Liang, Weidong Cao
Department of Electrical and Computer Engineering, The George Washington University, DC, US

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11044290

***Abstract*—Compute-in-memory (CIM) has shown significant potential in efficiently accelerating deep neural networks (DNNs) at the edge, particularly in speeding up quantized models for inference applications. Recently, there has been growing interest in developing floating-point-based CIM macros to improve the accuracy of high-precision DNN models, including both inference and training tasks. Yet, current implementations rely primarily on digital methods, leading to substantial power consumption. This paper introduces a hybrid domain CIM architecture that integrates analog and digital CIM within the same memory cell to efficiently accelerate high-precision DNNs. Specifically, we develop area-efficient circuits and energy-efficient analog-to-digital conversion techniques to realize this architecture. Comprehensive circuit-level simulations reveal the notable energy efficiency and lossless accuracy of the proposed design on benchmarks.**

I. Introduction

Compute-in-memory (CIM) holds significant promise for efficiently accelerating deep neural networks (DNNs) at the edge by bringing computation and memory closer together to reduce energy-intensive data movement that occurs in conventional von Neumann architecture [1]–[4]. Although numerous existing works have focused on building integer-based (INT-based) CIM macros [5]–[9] to improve the energy efficiency of quantized DNN models, there is growing interest in developing floating-point-based (FP-based) CIM macros to enhance the accuracy of high-precision DNN models, including both inference and training applications. Yet, most current FP CIM methods rely on digital implementation, leading to significant power consumption [10]–[12]. Novel approaches are needed to bridge this gap, ensuring both high accuracy and energy efficiency for increasing high-precision DNN applications.

This paper proposes a novel hybrid-domain FP CIM architecture to remarkably improve energy efficiency while maintaining lossless accuracy for high-precision DNN applications. The proposed architecture is based on a key observation that has been significantly overlooked in conventional designs of FP CIM macros. It is observed that the FP arithmetic can be intrinsically divided into two parts: (1) the computation-intensive multiplication (sub-MUL), contributing less than $1/4$ to FP products, and (2) the computation-light addition (sub-ADD), contributing more than $3/4$ to FP products, as elaborated in Section III-A. We harness this insight to strategically integrate both analog CIM (for energy-efficient sub-MUL) and digital CIM (for accurate sub-ADD) on a unified hardware substrate. This hybrid-domain CIM strategy, to the best of our knowledge, optimally combines the strengths of analog and digital CIM to achieve state-of-the-art energy efficiency by maintaining equivalent accuracy compared to fully digital baselines. The key contributions of the work are listed below.

- We propose a novel hybrid-domain FP CIM architecture based on static random-access memory (SRAM) that integrates both digital and analog CIM within a unified macro to accelerate high-precision DNN applications.
- We develop detailed circuit schematics and physical layouts to implement this architecture. We optimize the local computing logic with minimal area overhead and minimize the energy cost of analog-to-digital conversion.
- Experimental evaluations with comprehensive circuit-level simulations demonstrate the exceptional energy efficiency and accuracy of the proposed architecture compared to conventional fully digital baselines.

II. Background and Related Work

A. Floating-Point Arithmetic Primitives

FP arithmetic operations are crucial to high-precision DNNs. **FP format:** A general FP number in scientific notation is expressed as $f = (-1)^S \cdot 2^E \cdot 1.M$, where S ($S = 0$ or $S = 1$), E, and M ($M \in (0, 1)$) represent the sign, exponent, and mantissa (fraction) of the number, respectively. The '1' before M is a hidden bit that is not explicitly shown in the binary format of an FP number. E is the actual exponent, with an offset applied to the exponent encoded in the standard format. For example, in the IEEE 8-bit FP format (FP8, E4M3 or E5M2), S is 1-bit, E is 4(5)-bit, M is 3(2)-bit (Fig. 1(b)). **FP multiplication:** The multiplication of FP numbers is a straightforward process, involving exponent addition (①) and mantissa multiplication (②). For simplicity, the example below demonstrates this using two positive FP numbers.

$$(2^{E_0} \cdot 1.M_0) \cdot (2^{E_1} \cdot 1.M_1) = \underbrace{(2^{E_0+E_1})}_{①} \cdot \underbrace{(1.M_0 \cdot 1.M_1)}_{②}. \quad (1)$$

FP addition: However, the addition of floating-point (FP) numbers is more complex and can be expressed as follows:

$$2^{E_0} \cdot 1.M_0 + 2^{E_1} \cdot 1.M_1 = \underbrace{2^{E_{\max}}}_{①} \cdot \underbrace{\left\{ \begin{array}{l} \underbrace{2^{E_0-E_{\max}}}_{②} \cdot 1.M_0 \\ + \\ \underbrace{2^{E_1-E_{\max}}}_{②} \cdot 1.M_1 \end{array} \right.}_{③}. \quad (2)$$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:07:52 UTC from IEEE Xplore. Restrictions apply.

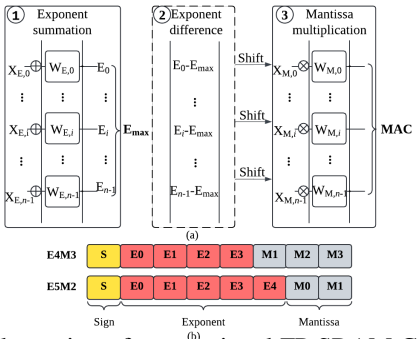


Fig. 1: (a) Illustration of conventional FP SRAM CIM architecture. $X_{E,i}$ and $W_{E,i}$ are the exponent parts of activation and weight. $X_{M,i}$ and $W_{M,i}$ are the mantissa parts of activation and weight. ①, ②, and ③ are circuit-level representations of the steps in Eq. (2), (b) FP8 format (E4M3 and E5M2).

The process begins with alignment, where the maximum exponent, $E_{\max} = \max\{E_0, E_1\}$, is identified among all exponents (①), followed by calculating the exponent difference, $E_{0(1)} - E_{\max}$, for each exponent (②). Next, the mantissa is shifted to the right based on this exponent difference and then summed (③). Finally, the result undergoes additional steps, such as truncation, to conform to the standard FP format.

FP vector-matrix multiplication: By generalizing Eq. (2) to the accumulation of n FP numbers, where each number, $2^{E_i} \cdot 1.M_i$ is assumed to be a product of a weight-activation pair, i.e., $(-1)^{W_{S,i}} \cdot 2^{W_{E,i}} \cdot 1.W_{M,i}$ and $(-1)^{X_{S,i}} \cdot 2^{X_{E,i}} \cdot 1.X_{M,i}$ based on Eq. (1) (i.e., $E_i = W_{E,i} + X_{E,i}$ and $1.M_i = 1.W_{M,i} \cdot 1.X_{M,i}$), the vector-matrix multiplication (VMM) of FP numbers is achieved. This operation is fundamental to FP DNN models. FP enables high-precision computations in DNN models, supporting the highest accuracy and best training quality [13]. Thus, efficient hardware acceleration for FP VMM is highly desirable.

B. Compute-in-Memory for FP DNNs

Previous work has explored CIM acceleration for FP DNN models using emerging non-volatile memory (NVM) devices [14], [15]. More recently, static random-access memory (SRAM) has become a cornerstone in the design of CIM systems for practical applications due to its exceptional accuracy in digital computing, along with superior performance, power efficiency, and area benefits [7], [10]–[12]. For instance, a previous study [12] introduced a reconfigurable FP/INT CIM processor capable of supporting both BFloat16 (BF16)/FP32 and INT8/16 in the same digital CIM macro. Another study [11] proposed dividing FP operations into high-efficiency intensive-CIM and flexible sparse-digital parts, based on the observation that most FP exponents cluster within a narrow range. Circuit-level techniques such as time-domain exponent summation mechanisms have also been employed to enhance the energy efficiency of CIM macros [10].

Nonetheless, these prior works share the common simplified architecture to accelerate FP VMM, as illustrated in Fig. 1(a), where all operations are performed in the digital domain. The process begins with the alignment of exponents by summing the exponent parts of the weight-activation pairs ($W_{E,i}$ and $X_{E,i}$, Step ①) and calculating the exponent difference between each sum, E_i, and the maximum exponent, $E_{\max}$ (Step ②). These exponent differences ($E_i - E_{\max}$) are then used to shift the mantissa parts of the activations, which are subsequently multiplied and accumulated (Step ③). As shown in Fig. 1(a), these steps align with the operations described in Eq. (2). Yet, our analysis reveals that by leveraging the inherent properties of FP arithmetic and the resilience of DNN models to minor computation errors, there is substantial untapped potential to use approximate computing (e.g., analog computing) to accelerate FP DNNs without sacrificing accuracy (Section III-A).

III. Hybrid-Domain FP CIM Architecture Design

A. Insight: Addition is More Important

Through a careful examination of FP arithmetic, we have identified a lightweight approach for implementing SRAM CIM macros in a hardware-friendly manner, enhancing energy efficiency while preserving the accuracy of FP DNNs. To illustrate this key concept, we decompose the conventional FP mantissa multiplication in Eq. (1) into two parts as follows:

$$\begin{aligned}(1.M_0 \cdot 1.M_1) &= (1 + M_0) \cdot (1 + M_1) \\ &= (\underbrace{1 + M_0 + M_1}_{\text{sub-ADD}} + \underbrace{M_0 \cdot M_1}_{\text{sub-MUL}}).\end{aligned} \tag{3}$$

Here, $(1 + M_0 + M_1)$ and $(M_0 \cdot M_1)$ are defined as mantissa sub-addition (sub-ADD) and mantissa sub-multiplication (sub-MUL), respectively. Based on this decomposition and given that $M_{0(1)} \in (0, 1)$, we analyze the significance of sub-MUL in mantissa multiplication [16], i.e.,

$$\frac{M_0 \cdot M_1}{(1 + M_0) \cdot (1 + M_1)} = \frac{1}{(1 + 1/M_0) \cdot (1 + 1/M_1)} \leq \frac{1}{4}. \tag{4}$$

Here, the '=' holds true if and only if $M_0 = M_1 \to 1$. Eq. (4) shows that for a single weight-activation pair, if the accuracy of sub-ADD is maintained, the *total computation error of mantissa multiplication (and thus the FP product) will not exceed* $1/4$ *of the true value, even with the aggressive removal of sub-MUL operations*. Our analysis reveals that while mantissa sub-MUL operations are computationally intensive, they often affect only a minority of FP products. In contrast, mantissa sub-ADD, though computationally lighter, constitutes the majority of FP products. Addition is also far more energy-efficient than multiplication; for instance, INT8 addition consumes only about 10% of the energy required for INT8 multiplication, as shown in previous work [17]. This key insight–'Addition is More Important'–guides our design strategy for FP SRAM CIM macro: **dedicating digital resources to ensure the precision of compute-light mantissa sub-ADD operations while leveraging energy-efficient analog computing for the more compute-intensive mantissa sub-MUL operations**.

B. Overall Macro Architecture

With the above key insight, we integrate analog CIM into existing well-established digital FP CIM architectures [7], [10] based on SRAM, achieving significant improvements in energy efficiency with minimal area overhead. Fig. 2 illustrates the proposed architecture, designed for FP8 DNN models (E4M3)

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:07:52 UTC from IEEE Xplore. Restrictions apply.

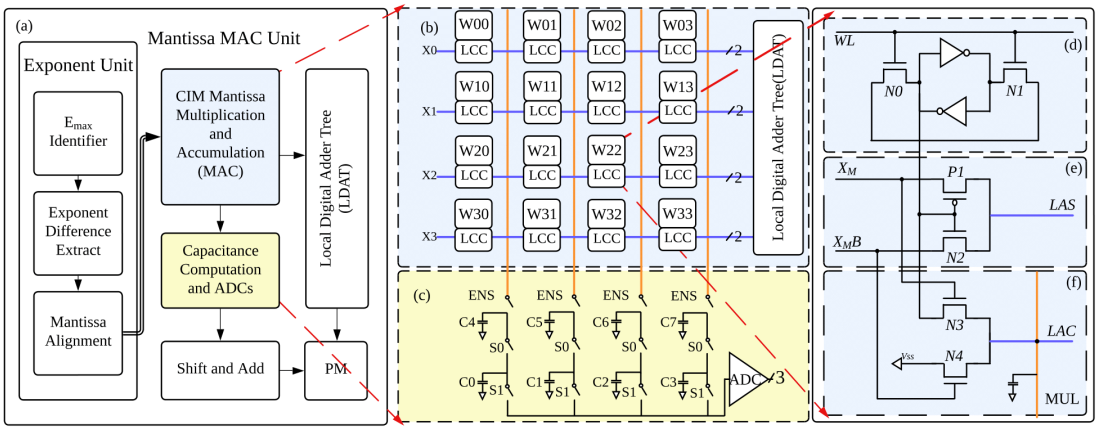


Fig. 2: (a) Architecture overview of the proposed hybrid-domain FP CIM macro, (b) Hybrid-domain FP SRAM CIM mantissa unit, (c) Switched-capacitor array with the Flash ADC, (d) 6T SRAM, (e) pseudo XOR gate, (f) pseudo AND gate, LAC for sub-add (blue line) and MUL for ACIM (orange line).

as an example. Note that FP8 demonstrates state-of-the-art energy efficiency in DNN applications while preserving accuracy [13]. The exponent unit comprises core components such as the exponent summation array, $E_{\max}$ identifier, exponent difference extractor, and mantissa alignment, in line with previous designs [10], [18]. The details are thus omitted here. We focus on the mantissa part. We introduce a novel hybrid-domain MAC unit to precisely manage mantissa sub-ADD in the digital realm while enhancing the energy efficiency of mantissa sub-MUL through analog computing.

Specifically, we demonstrate a hybrid-domain CIM mantissa MAC array with a 4×4 size as a proof of concept (Fig. 2(b)). This 4×4 array is ideally suited for a 4-bit weight mantissa in E4M3 FP8 formate, including a 1-bit sign indicator. Each bit of a weight mantissa $W_{M,i}$ is stored within the same row but across different columns of the 6T-SRAM cell array, ordered from the most significant bit (MSB) to the least significant bit (LSB). There are two key design innovations in this MAC array: (1) an area-efficient hybrid-domain local computing cell (LCC) is embedded into each memory cell to facilitate both sub-ADD and sub-MUL; (2) the analog partial sums of sub-MUL are accumulated into the analog domain to minimize analog-to-digital conversions. In the following subsections, we will introduce how sub-ADD and sub-MUL are enabled with these circuit-level design innovations.

C. Digital CIM – Sub-ADD

The sub-ADD operation is achieved through two pseudo-digital logic gates (a pseudo AND gate and a pseudo XOR gate) in the LCL. These gates, each composed of only two transistors to minimize area overhead, form a half-adder. Additionally, the pseudo AND gate is reused in a time-multiplexed manner for the sub-MUL operation, as shown in the next section. Using a bit-serial input, a 2-bit partial sum of the activation and weight mantissas is generated along each row in the jth cycle. When $X_{m,i}[j] = 1$, $\text{LAC}_{i,j} = W_{M,i}[j] - V_{\text{th}}$; when $X_{M,i}[j] = 0$, $\text{LAC}_{i,j} = V_{\text{SS}}$. Here, LAC represents the carry bit in the summation of the activation and weight mantissa, specifically for the jth bit of the mantissa part. Similiarly, pseudo XOR is equivalent to normal XOR gate in half adder. The 2-bit partial sum will be accumulated with the local adder to obtain the full sum of sub-ADD.

D. Analog CIM – Sub-MUL

Fig. 2(b) illustrates the sub-MUL operation, which is divided into two key steps: first, the pseudo AND gate in each cell performs the multiplication between the activation mantissa and weight mantissa; second, the resulting multiplication values from each cell are summed along each bit line.

1) Analog multiplication by reusing the AND gate: The pseudo AND logic gate obtains and sends the mantissa multiplication result to the capacitor computation unit through orange line on Fig. 2(f). The analog calculation relies on Kirchhoff's law: voltage on each GBLB decreases due to the discharge current from cells and increases with the charging current. The final voltage on each bit line reflects the net current of charged and discharged cells on each GBLB.

2) Analog accumulation of partial sums: We use the switched-capacitor array to efficiently accumulate analog partial sums of sub-ADD across bitlines. This mechanism reduces the required analog-to-digital conversions from four to one per input cycle, significantly enhancing energy efficiency. Assume the voltage of each GBLB$[i]$ is V_i and the capacitance is C_i, then we have $Q_i = V_i \cdot C_i$. All charges are collected first in the computation capacitance region as $Q_{total} = \sum_{i=0}^{3} V_i \cdot C_i$. When switch S1 is activated, charge sharing occurs at this stage, resulting in $V_o = (\sum_{i=0}^{3} V_i \cdot C_i)/(\sum_{i=0}^{3} \cdot C_i)$. Hence, V_o represents the 1-bit multiplication of the activation mantissa and the 4-bit weight mantissa.

Fig. 2(c) illustrates the multi-bit weight realization using charge sharing among computational capacitors. From the LSB to the MSB, the GBLB (orange line) connects to computation capacitors C_0, C_1, C_2, C_3 in a 1:2:4:8 capacitance ratio. Initially, all capacitors are pre-charged to V_{DD}. To maintain a total of 8 units of capacitance on each column, the compensation capacitors C_4, C_5, C_6, C_7 are configured with ratios of 7:6:4:0, respectively. After processing by the computation capacitors, the final voltage represents the multiplication of the 4-bit

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:07:52 UTC from IEEE Xplore. Restrictions apply.

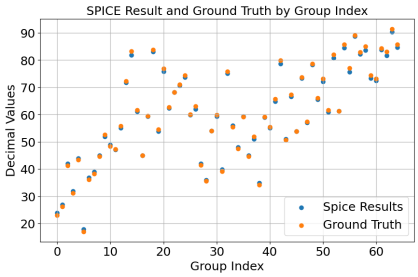


Fig. 3: The computation error across multiple combinations of 4-b inputs and 4-b weight

input with 1-bit weights. This voltage, representing a 4×4-bit multiply-accumulate (MAC) result, is isolated from the GBLB to prevent signal interference and converted to a 3-bit digital output by the Flash ADC.

The Flash ADC utilizes area-efficient sense amplifiers (SAs) instead of traditional analog comparators to reduce both area and energy consumption. Each 3-bit Flash ADC is composed of 7 SAs. In a 4×4 CIM array, the multiplication result voltage on each global bit line buffer (GBLB) may yield 4 possible values, resulting in 4×16 potential outputs, which would typically require a 6-bit ADC for full encoding. To optimize efficiency, we encode only the 3 most significant bits (MSBs). This selective encoding not only minimizes encoding area overlap due to nonlinearity but also reduces the impact on accuracy, ensuring effective performance with lower resource demands. Our simulation results in Section IV show that 3-bit ADC is sufficient to achieve loss-less accuracy for inference/training.

IV. Evaluation

A. Computation Accuracy

Since the exponent array is implemented in the digital domain without computation errors, we focus on presenting the computing accuracy of our mantissa MAC array. We compare the results from circuit-level simulations through the CIM macro and its ground truth which is obtained by simulating the computation flow through a Python script. We obtain the product of 4×4-b inputs and the 4×4-b weights. The computation error across multiple combinations of 4-b inputs and 4-b weights are shown in Fig. 3. It is small within 1.51% range. The result shows that the proposed hybrid-domain computing introduces minimal computation error.

We then apply the computed error to FP products based on FP arithmetic and inject this error into FP8 models to examine its effect on model accuracy. We evaluate accuracy across various FP DNN models using the MLPerf Edge Inference Benchmark Suite v4.0, which encompasses a diverse range of deep learning tasks. As shown in Table I, our proposed hybrid-domain architecture effectively maintains inference accuracy, with only approximately 1% variation observed for ResNet and BERT models, and virtually no accuracy loss for RetinaNet. This outcome underscores our macro's reliability in preserving system-level inference accuracy, likely due to the inherent robustness of these models.

TABLE I: Model Performance on Different Datasets.

Model	Dataset	Baseline Accuracy	Result
ResNet50	ImageNet	76.01% (top1 Acc)	75.68%
BERT-base	SQuAD v1.1	90.73% (f1_score)	89.49%
Retinanet	Open Images	0.3759 (mAP)	0.3709

B. Energy Comparison

We designed the proposed Mantissa MAC unit using 28 nm CMOS technology and compared its energy efficiency to a fully digital baseline, which includes a digital multiplier at each memory cell (enabled by a NAND gate), a local digital adder tree, an accumulator, and registers. Energy data for the digital baseline is derived from the quantitative model in [19]. Table II illustrates the comparison, showing our design achieves $1.53\times$ energy efficiency compared to the digital CIM.

TABLE II: Energy Breakdown Estimation (fJ/MAC).

Operation	Add/ Multi	Adder Tree	Accumulator/ ADC	Register	Total
sub-ADD	0.896	10.752	10.752	6.720	29.120
sub-MUL	1.792	8.064	7.04	5.376	22.272
Baseline	3.584	34.944	26.880	13.440	78.848
Efficiency	-	-	-	-	1.53×

C. Area Breakdown Estimation

We finally show the area efficiency of the proposed mantissa MAC unit. Each cell consists of a 6T-SRAM cell, a pseudo AND, and a pseudo XOR, totaling 11 transistors (4 PMOS and 7 NMOS). The SRAM cell and pseudo AND are symmetrical, resulting in a compact and efficient mantissa computing unit, as shown in Fig. 4(a). It leads to a 42.8% area overhead of the initial 6T SRAM cell. When expanded to configurations of two or four cells, the layout achieves even greater symmetry and space efficiency, as shown in Fig. 4(b). As noted, a traditional digital CIM macro, which uses NAND gates attached to each memory cell, employs a nearly identical number of transistors, thus resulting in minimal area variation between the two macros.

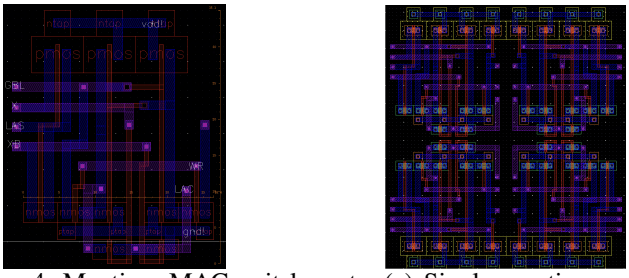


Fig. 4: Mantissa MAC unit layouts: (a) Single mantissa computing unit, (b) Layout of 4 cells.

V. Conclusion

We propose a novel hybrid-domain FP CIM architecture based on SRAM, integrating both digital and analog CIM elements. Detailed circuit designs were developed for the hybrid-domain mantissa MAC unit, and circuit-level evaluations demonstrate notable energy efficiency gains compared to fully digital baselines, with minimal area overhead and negligible accuracy loss.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:07:52 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] P.-C. Wu, J.-W. Su, L.-Y. Hong, J.-S. Ren, C.-H. Chien, H.-Y. Chen, C.-E. Ke, H.-M. Hsiao, S.-H. Li, S.-S. Sheu *et al.*, "A floating-point 6t sram in-memory-compute macro using hybrid-domain structure for advanced ai edge chips," *IEEE Journal of Solid-State Circuits*, 2023.

[2] J.-W. Su, Y.-C. Chou, R. Liu, T.-W. Liu, P.-J. Lu, P.-C. Wu, Y.-L. Chung, L.-Y. Hong, J.-S. Ren, T. Pan *et al.*, "A 8-b-precision 6t sram computing-in-memory macro using segmented-bitline charge-sharing scheme for ai edge chips," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 3, pp. 877–892, 2022.

[3] W. Cao and X. Zhang, "A/d alleviator: Reducing analog-to-digital conversions in compute-in-memory with augmented analog accumulation," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2023, pp. 1–5.

[4] W. Cao, Y. Zhao, A. Boloor, Y. Han, X. Zhang, and L. Jiang, "Neural-pim: Efficient processing-in-memory with neural approximation of peripherals," *IEEE Transactions on Computers*, vol. 71, no. 9, pp. 2142–2155, 2021.

[5] Q. Dong, M. E. Sinangil, B. Erbagci, D. Sun, W.-S. Khwa, H.-J. Liao, Y. Wang, and J. Chang, "15.3 a 351tops/w and 372.4 gops compute-in-memory sram macro in 7nm finfet cmos for machine-learning applications," in *2020 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2020, pp. 242–244.

[6] M.-E. Shih, S.-W. Hsieh, P.-Y. Tsai, M.-H. Lin, P.-K. Tsung, E.-J. Chang, J. Liang, S.-H. Chang, C.-L. Huang, Y.-Y. Nian *et al.*, "20.1 nve: A 3nm 23.2 tops/w 12b-digital-cim-based neural engine for high-resolution visual-quality enhancement on smart devices," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67. IEEE, 2024, pp. 360–362.

[7] J.-W. Su, Y.-C. Chou, and et al, "16.3 a 28nm 384kb 6t-sram computation-in-memory macro with 8b precision for ai edge chips," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, 2021, pp. 250–252.

[8] X. Sun, W. Cao, B. Crafton, K. Akarvardar, H. Mori, H. Fujiwara, H. Noguchi, Y.-D. Chih, M.-F. Chang, Y. Wang *et al.*, "Efficient processing of mlperf mobile workloads using digital compute-in-memory macros," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[9] J. Oh, S. K. Lee, M. Kang, M. Ziegler, J. Silberman, A. Agrawal, S. Venkataramani, B. Fleischer, M. Guillorn, J. Choi, W. Wang, S. Mueller, S. Ben-Yehuda, J. Bonanno, N. Cao, R. Casatuta, C.-Y. Chen, M. Cohen, O. Erez, T. Fox, G. Gristede, H. Haynie, V. Ivanov, S. Koswatta, S.-H. Lo, M. Lutz, G. Maier, A. Mesh, Y. Nustov, S. Rider, M. Schaal, M. Scheuermann, X. Sun, N. Wang, F. Yee, C. Zhou, V. Shah, B. Curran, V. Srinivasan, P.-F. Lu, S. Shukla, K. Gopalakrishnan, and L. Chang, "A 3.0 tflops 0.62v scalable processor core for high compute utilization ai training and inference," in *2020 IEEE Symposium on VLSI Circuits*, 2020, pp. 1–2.

[10] P.-C. Wu, J.-W. Su, and et al, "A 22nm 832kb hybrid-domain floating-point sram in-memory-compute macro with 16.2-70.2tflops/w for high-accuracy ai-edge devices," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 126–128.

[11] J. Yue, C. He, and et al, "A 28nm 16.9-300tops/w computing-in-memory processor supporting floating-point nn inference/training with intensive-cim sparse-digital architecture," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 1–3.

[12] F. Tu and et al, "A 28nm 29.2tflops/w bf16 and 36.5tops/w int8 reconfigurable digital cim processor with unified fp/int pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, 2022, pp. 1–3.

[13] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu *et al.*, "Fp8 formats for deep learning," *arXiv preprint arXiv:2209.05433*, 2022.

[14] Y. Luo and S. Yu, "Accelerating deep neural network in-situ training with non-volatile and volatile memory based hybrid precision synapses," *IEEE Transactions on Computers*, vol. 69, no. 8, pp. 1113–1127, 2020.

[15] M. Imani, S. Gupta, Y. Kim, and T. Rosing, "Floatpim: In-memory acceleration of deep neural network training with high precision," in *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, 2019, pp. 802–815.

[16] W. Cao, J. Gao, X. Xin, and X. Zhang, "Addition is most you need: Efficient floating-point sram compute-in-memory by harnessing mantissa addition," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[17] N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, M. Gottscho, and et al, "Ten lessons from three generations shaped google's tpuv4i : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1–14.

[18] P.-C. Wu, J.-W. Su, Y.-L. Chung, and et al, "A 28nm 1mb time-domain computing-in-memory 6t-sram macro with a 6.6ns latency, 1241gops and 37.01tops/w for 8b-mac operations for edge-ai devices," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.

[19] J. Sun, P. Houshmand, and M. Verhelst, "Analog or digital in-memory computing? benchmarking through quantitative modeling," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:07:52 UTC from IEEE Xplore. Restrictions apply.